

第 13 章

软件安全性测试

计算机的安全问题似乎每天都在出现。黑客、病毒、蠕虫、间谍软件、后门程序、木马、拒绝服务攻击都成了常见的术语了。每个计算机用户都遇到过这些麻烦的事情，在受到攻击后，丢失了重要的数据，浪费宝贵的时间在系统的恢复上。2005年1月14日，洛杉矶时报 Joseph Menn撰文，标题为“没有更多的互联网给他们”，披露许多人都受够了，他们受到挫折，他们愤怒，他们拨开连接——为了重新获得对其PC的控制。对于计算机产业的健康发展来说，这不是一个好现象。

因为这些原因，软件安全存在于每一个程序员的大脑中（如果要保住这份工作，至少应该这样），并且涉及到软件开发过程的每一个方面。软件测试联系到了另外一个领域，本章将介绍这些重要的和时尚的话题。

本章重点包括：

- 为什么有人想攻击计算机
- 哪些类型的攻击是常见的
- 如何与开发小组协作确认安全问题
- 为什么软件安全问题与软件缺陷是同一回事情
- 软件测试员在发现安全漏洞方面可以做哪些事情
- 作为新的领域的计算机取证与软件安全性测试有何关系

13.1 战争游戏——电影

计算机攻击首次进入公众视野的例子之一是1983年的电影战争游戏（WarGames）。这部电影中，Mathew Broderick的天才少年形象——David Lightman，使用他的IMSAI 8080家用计算机和一个300波特的音频Modem攻入了美国政府的北美防空联合司令部（NORAD）计算机系统。进入后，他偶然发现一个战争游戏叫做“全球热核战争”，他认为仅仅是个游戏，并和电脑系统玩起了该游戏，但哪知这是一个真实场景的模拟仿真，由此几乎引起了第三次世界大战。

David Lightman是如何获得访问权限的？很简单，他在自己的电脑上编了一个程序，顺序地从例如555-0000到555-9999进行拨号，并监听对方计算机Modem的应答。如果对方是个人在应答，计算机就挂机。他的程序是自动运行的，而程序运行的同时他在学校里创建了一个“热线”号码清单。放学后，David使用该清单上的有限号码逐一拨号，看哪一台计算机允许他登录。

在碰到北美防空联合司令部的计算机时，David经过了一些迂回曲折，发现了其中一个政

府程序员的名字以及该程序员去世的儿子的名字。David在登录密码输入区输入了该程序员儿子的名字——JOSHUA, 获得了对该军方“高度安全”的计算机系统的完全访问权限。

20年后, 技术已经变化了, 但是技巧和过程还是一样的。软件和计算机系统目前的互连更广泛, 不幸的是, 黑客更多了。也许有人会说电影战争游戏比电影里提到的又具有了新的意义, 即黑客在玩游戏, 同时软件产业则在与其进行一场战争。软件安全性, 更确切地说, 软件缺少安全性, 已经成为了一个巨大的问题。

驾驶攻击

随着在城域网中普及无线高保真(WiFi)网络, 经验丰富的黑客发现他们可以寻找开放的计算机网络, 就像Mathew Broderick在战争游戏电影里的天才少年形象一样。他们仅仅是带着笔记本电脑和廉价的“sniffer”开着车在城市的街道上兜圈子, 坐在企业建筑物的边上, 路过咖啡店或餐厅, 就可以搜索到未受保护的无线网络。这种技术从这部电影中得到了一个名字, 叫做“驾驶攻击”。

13.2 了解动机

在电影战争游戏的例子中, 黑客的动机在于好奇并且希望能使用一台比自己的功能强大得多的安全计算机。虽然他认为他的行为不是恶意的, 因为他并没有计划破坏系统, 但他的行为导致连锁的反应并带来了严重的结果。作为软件测试员很重要的一点是要了解为什么有人要攻击你的软件。

了解动机能帮助软件测试员考虑到测试的软件中有哪些安全方面的漏洞。

安全产品是指产品在系统的所有者或管理员的控制下, 保护用户信息的保密性、完整性、可获得性, 以及处理资源的完整性和可获得性。

www.microsoft.com/technet/community/chats/trans/security/sec0612.msp

安全漏洞是指使产品不可行的缺陷——即使是正确地使用产品时——来防止攻击者窃取系统的用户权限、调节操作、破坏数据, 或建立未授权的信任。

<http://www.microsoft.com/technet/archive/community/columns/security/essays/vulnrbl.msp>

黑客:

精通计算机编程和使用的人, 电脑玩家。

使用编程技能来获得对计算机网络或文件的非法访问的人。

www.dictionary.com

黑客想获得系统访问权限的5个动机是:

- **挑战/成名。**最简单和良性的黑客攻击是, 纯粹为了挑战性的任务或在黑客同行中形成成功者的威望而攻进一个系统。在这些行为中并没有更险恶的意图, 驾驶攻击就是这类行为。虽然看起来这没有多大的问题, 但是想想如果一个开锁匠为练习开锁技能每天晚上在邻居家随意开锁, 然后向他的朋友吹嘘哪些家的锁很容易开的话, 没有人会感到安全——准确地说确实是这样。
- **好奇。**下一个动机是好奇。在这种心理下, 黑客不会停止在仅仅获得访问权限上。一旦进入后, 他会进一步去看里面有什么。好奇就是动机, 黑客会在系统中找有兴趣的东西。一个软件系统可能有安全漏洞使黑客获得访问权限(出于挑战/成名的动机), 但是其安

全度仍然足以阻止黑客对任何有价值数据的进一步访问。

- **使用/借用。**这个动机下黑客的行为就不仅仅是攻击和进入了。实际上黑客为自己的目的会尝试使用系统。前面谈到的电影战争游戏就是使用/借用的例子。一个现实的例子就是当家用PC被E-mail病毒攻击后，会使用存储在PC机上的E-mail地址来发送更多的病毒邮件。黑客使用许多计算机的分布式处理能力可以比仅仅使用自己的计算机能完成更多的事情。另外，黑客使用这些被黑的计算机进行攻击能更有效地掩饰其痕迹。
- **恶意破坏。**一想到恶意破坏，请记住三个D：丑化（Defacing），破坏（Destruction）和拒绝服务（Denial of Service）。丑化是改变网站的外观来展示黑客的意见和想法——见图13-1。破坏以删除或修改存储在系统上的数据为表现形式。一个例子是某个大学生改变他的分数或删除期末考试成绩。拒绝服务是阻止或妨碍被黑的系统执行正常的操作。一个例子是使用大量的传输流攻击一个电子商务站点，使其不能处理正常的传输，阻止客户下单购买东西。更糟的是，黑客可能使系统崩溃，其结果是数据的丢失和数天的停机。
- **偷窃。**最严重的黑客攻击行为可能就是偷窃了。其动机是找出可以使用和出卖的有价值的东西。信用卡号、个人信息、商品和服务，甚至登录ID号和E-mail地址，所有这些对于黑客来说都有用。2003年，一个24岁的计算机黑客获得访问权限并偷盗了9千2百万个AOL登录ID号。这些ID号后来向信息兜售者卖了几次，总售价达10万美元！对于几天的工作就搞定来说还不错。

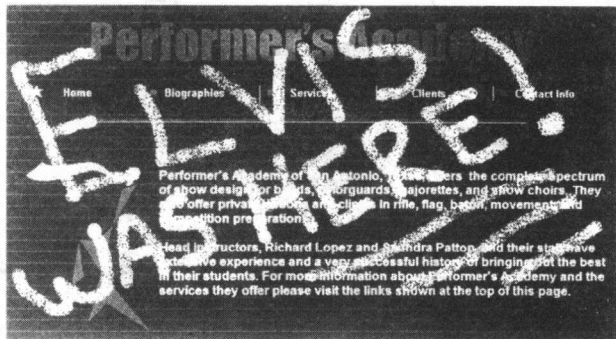


图13-1 丑化一个站点是黑客进行的破坏类型之一

感到害怕了吧？可能。每天都有无数的黑客尝试攻击那些理论上安全的系统。而且，很多都成功了。现在知道了为什么黑客想要攻击系统，下面我们将继续讨论软件安全性测试，归纳出软件测试员在协助开发小组创建安全的软件产品上可以做的工作。

13.3 威胁模式分析

在Michael Howard和David LeBlanc所著的一本优秀书籍《Writing Secure Code》（微软出版社，2003年，第2版）中，讨论到一个过程，叫做威胁模式分析（threat modeling），用于评估软件系统的安全问题。把威胁模式分析看做第6章“检查代码”中讨论的正规审查的一个变形方式。在这个过程中，目的是由评审小组查找产品特性设置方面可能会引起安全漏洞的地方。根据这些信息，小组可以选择对产品做修改，花更多的努力设计特定的功能，或者集中精力测试潜在的故障点。最终，这种理解会使产品更加安全。

注意 除非产品开发小组中的每个人——是指每一个人：项目经理、程序员、测试员、技术文档写作员、市场人员、产品支持——都理解和认同可能存在的安全威胁，否则小组不可能开发出安全的产品来。

执行威胁模式分析并非软件测试员的责任。这个责任应该落到项目经理的任务清单上，并且项目小组每个成员都要参与。基于此，我们仅讨论过程的共性。需要了解威胁模式分析的更详细信息，可以访问msdn.microsoft.com/library，搜索“threat modeling”。

图13-2显示了一个虚构的复杂电子商务系统的物理拓扑图。注意，该系统具有多种用户访问方式。用户可以通过PDA访问系统，通过无线连接访问系统，通过标准PC访问系统。系统连接到Internet并且具有来自不同厂家的服务器软件。系统在一个通过Internet连接可以直接访问的数据库里保存支付信息，这个系统的初始设计要求进行全面的威胁模型分析来确保投入运行后的安全。

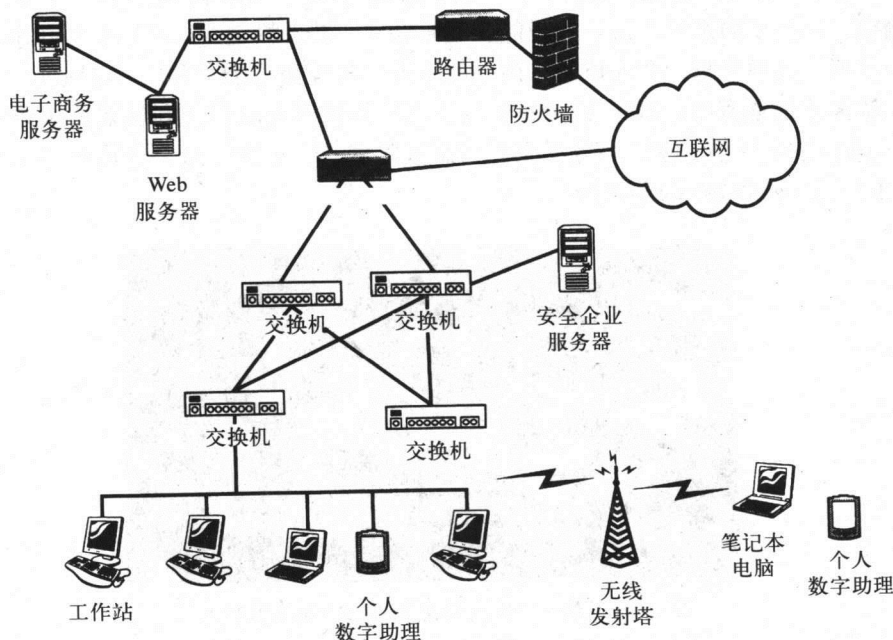


图13-2 一个复杂的系统要求进行全面的威胁模型分析来确认安全漏洞

图13-2中所示的软件和系统的复杂程度跟大家正在开发的软件和系统比起来是否简单或复杂在这里其实并没有任何的关系。威胁模型分析过程的步骤都是一样的。以下的步骤摘自msdn.microsoft.com/library。

- **构建威胁模型分析小组。**我们已经谈到了该问题。除了标准的小组成员外，小组中加入一个具有深厚的软件安全背景的人至关重要。对一个规模小的小组来说，方式上可能是在设计阶段让外部的咨询人员介入。对于大公司，这类人员往往来自于从一个项目转到另外一个项目、为项目提供专业咨询的一群专家。对于小组来说，重要的一点是了解他们的最初目标不是解决安全问题，而是确定安全问题。在后期可以举行一些由小规模的特定制团队参加的会议（例如，由几个程序员和测试员参加的会议），以隔离安全威胁，设计解决方案。

- **确认价值。**考虑系统所有的东西对于一个入侵者说价值有多大。系统是否有客户的个人信息？是否有信用卡号码？系统计算能力如何？是否有人想偷偷利用你的系统来向你的客户发送商业信息？如果你的公司网站被黑掉，不管是丑化还是替换，是否会带来业务的损失？
- **创建一个体系结构总体图。**在这个步骤中要确认计划用在软件中的以及如何实现互连的技术。小的威胁模型分析小组会创建一个体系结构图表示出主要的技术模块和它们之间如何通讯。创建体系结构图一个重要的方面是，确认在不同技术和其证明之间的信任边界（trust boundaries）以及为了访问数据必须发生的授权。
- **分解应用程序。**这是一个格式化的过程，用来确认数据所在位置以及如何通过系统。理想情况下，该步骤应基于设计的数据流图和状态转换图。如果这些图不存在，需要创建这些图。想想作为容器的数据以及哪些使容器安全。进入容器来查看数据的手段是什么？数据加密了吗？口令保护了吗？
- **确认威胁。**一旦完全理解了所有的部分（价值、体系结构、数据），威胁模型分析小组可以转向确认威胁。每一个部分都应该考虑为威胁目标，并且应假设他们会受到攻击。想想是否每一部分都可能被不正确地看待。每一部分是否可能被修改？黑客是否能阻止授权用户使用系统？是否有人能获得系统的访问权限并控制系统？
- **记录威胁。**每个威胁都必须用文档记录，并且应进行跟踪以确保其被解决。文档是一种简单方式，用于描述威胁（用一句或更少）、目标、攻击可能采用的方式、系统用于防御攻击有哪些反制手段。在第19章“报告发现的问题”，将讲述如何跟踪发现的软件缺陷。使用相同的系统可以帮助小组管理发现的安全威胁。
- **威胁等级评定。**最后，理解并非所有的威胁生来就平等这一点很重要。软件数据可能用国防部128位加密算法保护，用超级计算机需要20多年才能破解。即便使用如此安全等级保护的数据会受到安全威胁吗？当然可能。这种威胁等级比一个连接到管理员的计算机的自动应答电话modem更高吗？可能不会。威胁模型分析小组需要确定每个威胁的等级。一个简单的办法是，使用在msdn.microsoft.com/library上“提高Web应用安全”那一章中定义的恐怖公式（DREAD Formula）。DREAD表示：
- **潜在的损害**——如果这部分被黑了，损害有多大？（物理上、金钱上、整体性上等）
- **可反复性**——黑客不间断利用漏洞的容易度如何？每次尝试都能成功？还是100次尝试成功一次？或1百万次尝试成功一次？
- **可利用性**——获得对系统或数据访问的技术难度有多大？是可以通过互联网用电子邮件发送，或用几行简单的basic写的宏代码，还是需要具有很专业编程技能的人员来实现？
- **受影响的用户**——如果黑客成功入侵，有多少用户会受到影响？是单个用户，还是像偷盗AOL屏幕上的用户名，有9千2百万用户？
- **可发现性**——黑客发现漏洞的可能性有多大？一个保密的“后门”登录口令可能不会被发现，除非心怀怨恨的雇员被解雇后把这些信息贴到Web上。

简单地用1表示低，2表示中等，3表示高，并将此应用到以上描述的五类评估分析的每一类，然后加起来，获得5到15之间的一个值，该值就可以以数值的方式来评估每一个威胁的安全等级。据此，小组可以首先计划对最严重的问题进行设计和测试，然后在时间允许时继续对其他低级的威胁进行设计和测试。第19章将讲述评估软件缺陷的更多内容以及确认软件缺

陷被修改的一些技术。

13.4 软件安全是一项功能吗？软件漏洞是一个缺陷吗

到目前为止，我们都希望得出这样的结论，即软件安全可以简单地看做是软件产品或系统的另外一项功能。大多数人可能都不愿意其个人财务数据被黑客公开。他们可能认为他们的电子表格软件能保护个人私有信息是一项必须的功能。该功能是可以使软件成为质量优良的产品的一个功能（记得第3章“软件测试的实质”中我们讨论的质量和可靠性吗？），如果软件“失败”并让黑客看到了个人的银行资金情况和信用卡信息，大多数用户会简单和绝对地将其视为软件缺陷。

提示 这里，再重述一次第1章“软件测试的背景”关于缺陷的定义：

- 1) 软件未实现产品说明书要求的功能。
- 2) 软件出现了产品说明书指明不应该出现的错误。
- 3) 软件实现了产品说明书未提到的功能。
- 4) 软件未实现产品说明书虽未明确提及但应该实现的目标。
- 5) 软件难以理解、不易使用、运行速度慢，或者软件测试员认为最终用户会认为不好。

然后，再重新看看我们在第5章“带上眼罩测试软件”中讨论的通过性测试和失效性测试：

在进行通过性测试时，实际上是确认软件至少能做什么，而不会考验其能力。软件测试员并不需要想尽办法让软件崩溃，仅仅运用最简单、最直观的测试用例即可。

确信软件在普通情况下正确运行之后，就可以采取各种手段搞垮软件来找出软件缺陷了。

软件测试员可能会负责测试软件的整个安全性，或者可能仅仅负责测试被分配测试的功能是安全的。不管怎样，软件测试员需要考虑到软件缺陷的以上5个定义。软件测试员不需要拿到一份清楚明白地定义软件安全性是如何实现的产品说明书。软件测试员也不能假设威胁模型分析是完全和准确的。基于这些原因，软件测试员需要带上“失效性测试”的帽子，像黑客一样攻击被测试的软件——假定每一项功能都有一个安全漏洞，并且作为测试员，这时你的工作是发现并利用它。

技巧 测试安全缺陷是失效性测试行为，也常常覆盖产品中没有被完全理解和说明的部分。

13.5 了解缓冲区溢出

要在一章甚至一本书中，充分全面地讲述攻击软件产品所有可能的方法是不可能的。总地看来，家庭无线网络共享的电子表格与多玩家的网络视频游戏或一台国防部的分布式计算机系统是相当不同的。操作系统和其他技术各有特点，因而其安全漏洞也各不相同。但是有一个共同性的问题，即在任何软件产品中都有一个安全问题——缓冲区溢出。

在第6章的通用代码审核清单中，我们了解到数据引用错误——即使用没有被正确申明和初始化的变量、常数、数组、字符串或记录引起的缺陷，缓冲区溢出就是这种缺陷。这是由

于编程质量低劣，加上许多编程语言（如C、C++）都缺少安全字符串处理函数，这样就会带来这些问题。考虑清单13-1给出的简单C代码。

清单13-1 简单缓冲区溢出的例子

```
1: void myBufferCopy(char * pSourceStr) {  
2:     char pDestStr[100];  
3:     int nLocalVar1 = 123;  
4:     int nLocalVar2 = 456;  
5:     strcpy(pDestStr, pSourceStr);  
...  
6: }  
7: void myValidate()  
8: {  
9: /*  
10: Assume this function's code validates a user password  
11: and grants access to millions of private customer records  
12: /*  
13: }
```

看出问题没有？输入字符串pSourceStr的长度是不知道的。目的字符串pDestStr的长度是100字节。如果源字符串的长度大于100会发生什么事情？就像代码中写的一样，源字符串会copy到目的字符串中，不管长度有多少。如果源字符串超过100个字节的长度，就会填满目的字符串，并且会继续覆盖本地变量的值。

然而糟糕的是，如果源字符串足够长，就可能会覆盖掉函数myBufferCopy的返回地址并进而覆盖函数myValidate()执行代码的内容。在这个例子中，一个有点能力的黑客会输入一个超级长的口令，用手写的汇编代码替代数字和文字的ASCII字符串，并覆盖原本执行口令验证的函数myValidate——这就可能获得访问系统的权限。突然之间，第6章讲到的代码审核具有了全新的含义。

注意 这是一个极其简单的缓冲区溢出的例子，用于演示潜在的问题。虽然哪些数据和代码被覆盖到，甚至被全部覆盖执行，仅依赖于编译器和CPU。但是，黑客是知道的。

由于字符串的不正确处理引起的缓冲区溢出是目前为止最为常见的一种代码编写错误，其结果是导致安全漏洞，不过第6章中描述的任何错误等级都是潜在的问题。作为软件测试员，其工作是尽可能早地发现这些类型的缺陷。虽然代码评审可能会在开发周期的早期发现这些缺陷，但是还有更好的方法——即在第一时间和地点就防止它们发生。

13.6 使用安全的字符串函数

在2002年，Microsoft开始主动确认通用C和C++函数中容易引起缓冲区溢出的编码错误。这些函数自身并不差，但是要安全地使用它们，需要在程序员这边进行更为深入的错误检测。如果忽略了这种错误检测（经常发生此类事情），代码就会有安全漏洞。衡量一下这种疏忽带来的风险，最好还是开发或改进一组新的函数，即用强壮、完全测试过的、文档齐全的新函数集替代这些容易引起问题的函数集。

这些新的函数，叫做安全字符串函数（Safe String Functions），在Microsoft的Windows

XP SP1版本、最新的版本Windows DDK和平台SDK中已经具有。常用的操作系统、编译器，处理器也具有其他很多实现了安全字符串的商用的或免费的库。

以下列出的几条解释了使用新函数的好处（摘自msdn.microsoft.com/library的文章Using Safe String Functions）:

- 每个函数接收目标缓冲的长度作为输入。这样函数就能确保在写入时不会超过缓冲区的长度。
- 函数空字符中止所有的输出字符串，即使操作截断了预计的结果。在返回字符串上进行操作的代码就能安全地假定其会最终遇到空字符——表示字符串的结束。在空字符前的数据是有效的，并且字符不会无穷尽地延长。
- 所有函数返回一个NTSTATUS值，该值只有一个可能成功的代码。调用函数能轻易地确定函数的执行是否成功。
- 每个都提供版本。一个支持单字节的ASCII字符，另一个支持双字节的Unicode字符。记住在第10章“外国语言测试”中讲到的，要支持多种外国语言的字符或象形文字，字符必须占用超过一个字节的空。

表13-1显示了不安全函数以及其替代函数的清单。项目小组在进行代码评审或者白盒测试时，注意关注不安全的函数以及其使用。显然，小组程序员应该使用安全版本的函数。但是如果没有使用，代码评审就要更加严格地执行，确保任何可能的安全漏洞都被发现并解决。

表13-1 旧的“不安全”C字符串函数和其新的“安全函数”

旧的“不安全”函数	新的“安全”函数	目 的
strcat wcscat	RtlStringCbCat RtlStringCbCatEx RtlStringCchCat RtlStringCchCatEx	Concatenate two strings.
strncat wcsncat	RtlStringCbCatN RtlStringCbCatNEx RtlStringCchCatN RtlStringCchCatNEx	Concatenate two byte-counted strings, while limiting the size of the appended string.
strcpy wcsncpy	RtlStringCbCopy RtlStringCbCopyEx RtlStringCchCopy RtlStringCchCopyEx	Copy a string into a buffer.
strncpy wcsncpy	RtlStringCbCopyN RtlStringCbCopyNEx RtlStringCchCopyN RtlStringCchCopyNEx	Copy a byte-counted string into a buffer, while limiting the size of the copied string.
strlen wcslen	RtlStringCbLength RtlStringCchLength	Determine the length of a supplied string.
sprintf swprintf _snprintf _snwprintf	RtlStringCbPrintf RtlStringCbPrintfEx RtlStringCchPrintf RtlStringCchPrintfEx	Create a formatted text string that is based on a format string and a set of additional function arguments.
vsprintf vswprintf _vsnprintf _vsnwprintf	RtlStringCbVPrintf RtlStringCbVPrintfEx RtlStringCchVPrintf RtlStringCchVPrintfEx	Create a formatted text string that is based on a format string and one additional function argument.

JPEG病毒

比一幅图片更安全的是什么？总的看来，应是数据，而不是可执行代码。但这个错误的假设在2004年9月被推翻，这时一个病毒被发现嵌入到几幅黄色图片中并上传到一个互联网新闻组上面。在查看图片时，病毒下载到用户的PC机上面。没有人认为这是可能的，但确实发生了。导致这个问题的原因在于对缓冲区溢出的利用。

JPEG文件格式，除了存储图片数据外，还存储嵌入的注释和评论。许多编辑图片的软件包使用这种格式来注释图片——如“我们家在沙滩”、“出售住宅”等。这些注释域以十六进制值0xFFFE开头，接下来是两个字节的值。该值说明注释的长度，加上两个字节（注释域长度）。使用这种格式，不超过65533个字节长度的注释都是合法的。如果没有注释，区域的值就应该是2。问题在于如果此值为非法的1或0，就会发生缓冲区溢出。

解释JPEG数据格式并将其转变为可见的图片的程序在读注释前，将文件长度减去2，变为正常长度。解释程序的代码可以处理正整数，而把负整数2当做正的4GB。这样下面4GB长的“注释”内容就被读入，从而不正确地覆盖了有效的数据和程序。如果“注释”数据被精心地构造、编码、编译，就可能用来获得系统的访问权限。Microsoft曾经发布过一个关键更新，针对系统所有加载和查看JPEG图片的组件。

13.7 计算机取证

到目前为止，我们从积极的出发点讨论了软件的安全问题。我们看待软件安全的观点是：黑客可能利用软件，发现安全漏洞，利用漏洞来达到访问数据或控制系统的目的。另外一个观点是：要达到此目前并不需要这么困难。有时，对于那些知道数据保存位置的人来说，要查看数据可以说是信手拈来。

这方面的第一个例子就是在浏览网页时我们都熟悉的一些特性。图13-3表示通过一个IE浏览器下拉地址框显示出最近访问过的站点的历史清单的例子。对大多数用户来说这不是个问题；实际上还很有用，可以不需要重新输入URL全名就可以很快地回到访问过的站点。但是，如果这个屏幕是来自一个排队公共使用的终端上的话，会怎样呢？排在你后面的人就可能知道你看到的内容，仅仅是单击一下鼠标。

注意 用户变更时未被删除的保留数据叫做潜在数据。潜在数据是潜在的安全漏洞，需要在小组采用的任何威胁模型分析中进行讨论。也许这些数据不会被看成是产品的问题，也许会被看成是一个大问题。

另外一个潜在数据的例子是Google工具条的自动填充功能，如图13-4所示。该功能允许存储姓名、地址、电话号码、e-mail地址等信息，可以在显示出一个空的表单时（例如电子商务的订货网页），仅通过一个单击就可以用此信息把所有的部分都填上。然而，在测试产品的安全性时，测试员需要从用户的角度考虑，以确定该项功能是否需要对数据进行隐藏或删除，使其他人看不到这些数据。

潜在数据的更复杂的例子是由计算机安全专家用来发现可以用做犯罪调查的证据。当数据写到磁盘上时，是以块的方式写的。这些块叫做扇区，其大小取决于操作系统的类型。MSDOS/Windows使用512字节的扇区。根据所使用的文件系统，扇区组合成簇。Windows的FAT文件系统使用2048字节大小的簇，每一个簇由4个512字节大小的扇区构成。

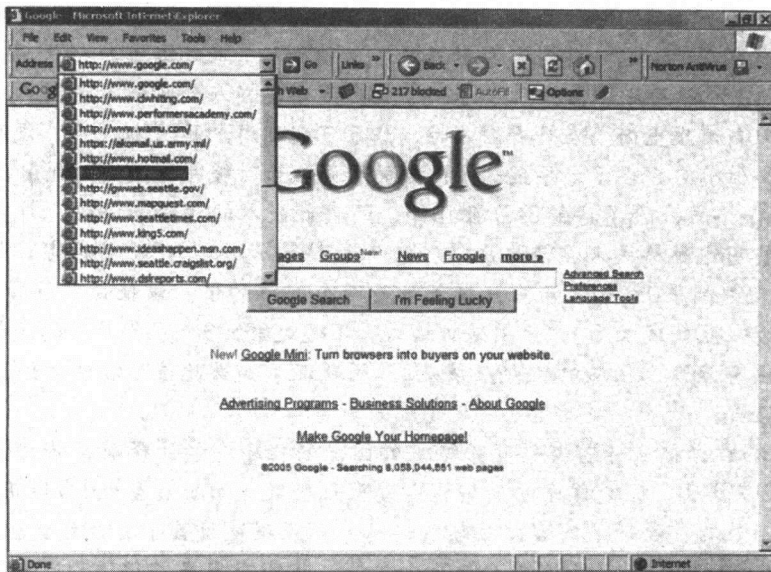


图13-3 访问过的站点清单可能是个安全漏洞

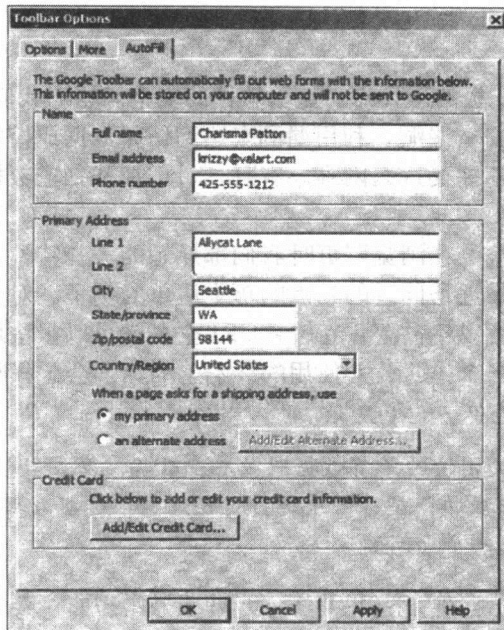


图13-4 Google工具条的自动填充功能存储的信息用于快速填写web表单。是个安全问题吗

图13-5显示了当一个文本文件readme.doc被写入到磁盘的两个簇上时的结构情况。文件大小为2200个字节，由从第1扇区到第5扇区中点的白色区域表示出来。所以，如果文件是2200个字节大，从第5扇区到第8扇区的灰色区域的内容是什么呢？这些信息就是潜在数据（latent data）。

如果文件有2200个字节大，会占用4.3个512字节的扇区（ $2200/512=4.3$ ）。在第5扇区，后半部分空间的数据叫做RAM损耗（RAM slack），因为这些区域的数据信息是在文件被创建时曾驻

留在内存中的数据信息。可能什么都没有，也可能是管理员的口令或信用卡账号。虽然没有办法知道内容，但是知道的是除了文件中的数据外，计算机的内存中的数据被写到了磁盘上。

余下的灰色区域，从第6扇区到第8扇区，叫做磁盘损耗 (disk slack)。磁盘损耗存在的原因是因为文件系统以一个2048个字节的簇来写磁盘，而我们的文件只能够填充其中两个簇的部分区域。在磁盘损耗位置保存的数据是该文件被写入前的数据。可能是其他文件的残余数据，或者是一个以前的、更长的readme.doc文件。这些潜在的数据可能是个开头，或者包含被特意删除的文件，或者是很保密的信息。

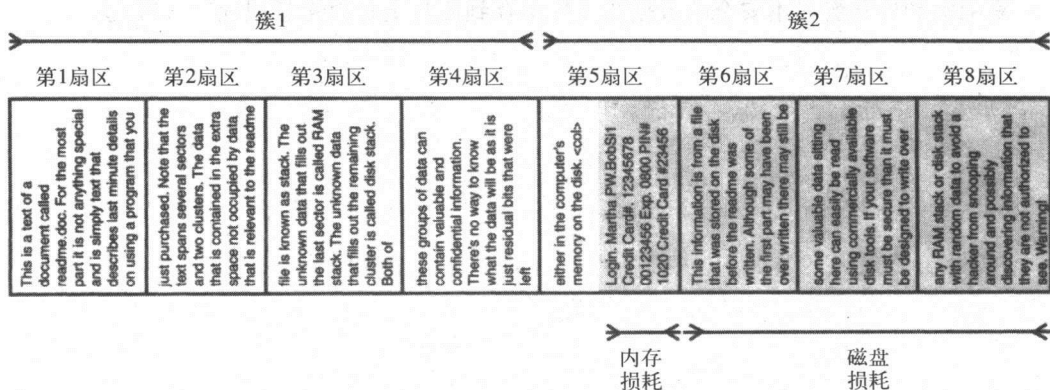


图13-5 readme.doc文件中的数据并非是唯一写到磁盘上的数据

注意 虽然这个例子使用磁盘驱动器来说明潜在数据的概念，但是RAM损耗和磁盘损耗带来的安全问题同样出现在可读写CD、DVD、内存卡，以及事实上任何种类的存储介质上。

13.8 小结

从本章中得到的一个体会应该是：没有计算机系统是安全的。我们应该假定这些计算机系统会一直处于受攻击的状态，攻击者或者想控制计算机，或者想访问到上面的数据。为帮助设计一个安全的系统，在产品设计的最开始就必须注意安全的问题。测试员仅能测试软件的安全性，软件安全必须先计划、评审、设计，然后才是测试。使用具有反复性的软件开发过程，例如第2章“软件开发的过程”详细分析的螺旋模式，可以确保在整个开发过程中安全的问题被再次提到。

跟踪最新的计算机安全问题的最有参考价值的网站是www.securityfocus.com。如果在负责软件安全的测试，或者甚至没有，定期浏览该网站是个好办法，看看黑客们在干什么，以及他们是如何破坏大大小小的系统的。

下一章将讲述另外一个时髦的话题，会用到目前所学的所有技术，包括软件安全性测试——即网站测试。

13.9 小测验

以下是帮助读者加深理解的小测验。答案参见附录A——但是不要偷看！

- 1. 在电影《WarGames》中攻破北美防空联合司令部（NORAD）的计算机系统背后的动